Tema 01: Punteros y Memoria Dinámica

METODOLOGÍA DE LA PROGRAMACIÓN

GRADO EN INGENIERÍA INFORMÁTICA

DECSAI

UNIVERSITAS GRANATENSIS · 1531 · PLUS ULTRA

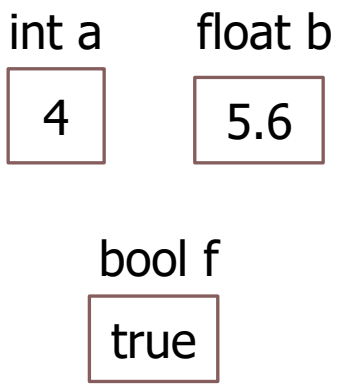
Índice

- Tipo de dato puntero.
- Memoria dinámica vs memoria estática.
- Vectores y matrices con memoria dinámica.
- Ejemplos de estructuras de datos simples.

Tipo de dato puntero

Variables vs punteros

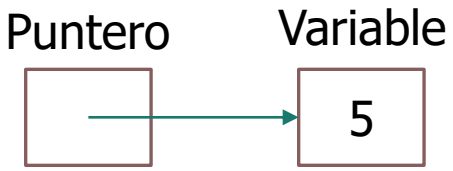
- Hasta ahora siempre hemos trabajado con variables.
- Recordemos:
  - Una variable almacena un valor.
  - Cada variable tiene un determinado tipo (entero, flotante, booleano, etc …).
  - Cada variable tiene un identificador único que usamos para trabajar con ella en programación.



Tipo de dato puntero

Punteros

- Contienen direcciones de memoria.
- A partir de las direcciones de memoria se puede acceder a los valores.
- Un puntero contiene la dirección de una variable que contiene un valor específico.
- **Variable**: referencia directa.
- **Puntero**: referencia indirecta.



Tipo de dato puntero

Declaración de un puntero

<tipo de dato> * identificador;

```
#include<iostream>

using namespace std;
int main(){
    int * puntero;
    cout << "El puntero apunta a:" << puntero << endl;
}
```

- Ejemplos:
  - bool * comprobante;
  - float * numeroReal;
  - int * contador;
  - float * uno, * dos;
  - int * uno, nopuntero;

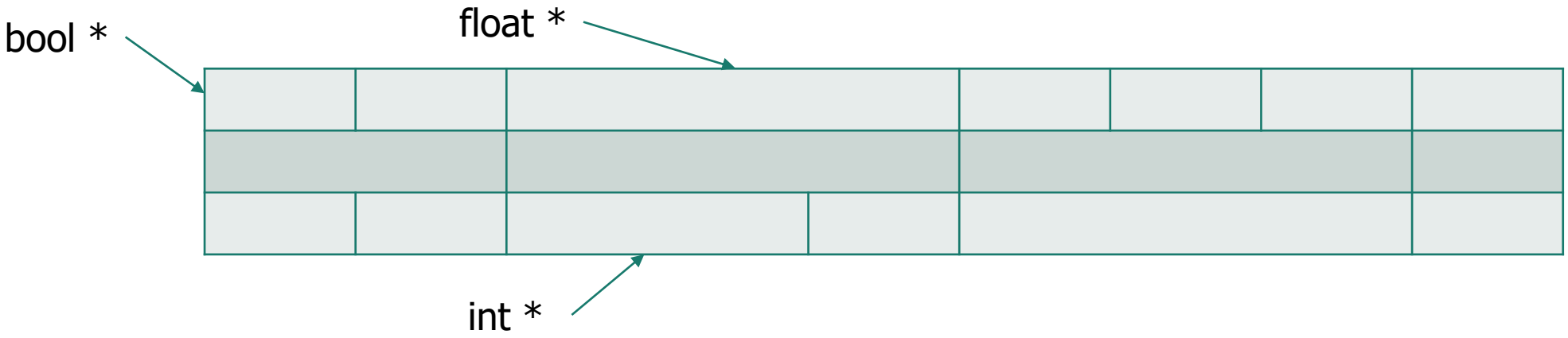
```
El puntero apunta a:0x401a5b

Process returned 0 (0x0)   execution time : 1.566 s
Press any key to continue.
```

Las direcciones de memoria dependen del compilador y el sistema operativo

Tipo de dato puntero

Importancia de indicar el tipo

- Cada tipo de variable ocupa valores diferentes en memoria.
- Cada puntero declarado apunta a un espacio en memoria de donde se espera poder leer un dato de un tipo específico.



Tipo de dato puntero

Importancia de indicar el tipo

- Tratar de asignar a un puntero de un tipo una dirección de memoria de otro tipo genera error en tiempo de compilación.
- Es importante, por tanto, que todos los tipos de los punteros sean iguales a los tipos de datos que les estamos asignando.
- Incluso float y double dan error:

```
#include<iostream>

using namespace std;

int main(){
    double * puntero;
    float a = 3;
    puntero = &a;

    cout << *puntero << " " << a << endl;
}
```

```
E:\pruebaco...   8   error: cannot convert 'float*' to 'double*' in assignment
```

Tipo de dato puntero

Inicialización de punteros

¡0 es el único valor que se puede asignar directamente a un puntero!

- Cuando inicializamos un puntero, la dirección a la que apunta suele estar vacía.
- Es bueno inicializarlos a 0, NULL o nullptr hasta que les demos algún uso.
- Podemos usar if (puntero==0) para detectar aquellos punteros no asignados si los inicializamos.

```
int * puntero2 =0;
cout << "El puntero apunta a:" << puntero2 << endl;
if(puntero2 == 0){
    cout << "no ha sido usado" << endl;
}
```

```
El puntero apunta a:0
no ha sido usado

Process returned 0 (0x0)   execution time : 0.044 s
Press any key to continue.
```

Tipo de dato puntero

DECSAI

Operadores para punteros

- **Operador de dirección &**
  - Devuelve la dirección de memoria de una variable.
  - Sirve para asignar la dirección de una variable específica a un puntero.
- **Operador de indirección ***
  - Devuelve el valor almacenado en una dirección de memoria o puntero.

Tipo de dato puntero

DECSAI

Uso de operadores relacionados con punteros

- Operador de dirección

<dirección de memoria>= & <variable>

& y * son inversos entre sí

- Operador de indirección

<valor de la variable>= *<puntero>

Tipo de dato puntero

Ejemplo de uso de operadores

```
#include<iostream>

using namespace std;
int main(){
    int * puntero;
    int y = 5;
    puntero = &y;
    cout << "El valor al que apunta puntero es:" << *puntero << endl ;
}
```

```
El valor al que apunta puntero es:5

Process returned 0 (0x0)   execution time : 1.372 s
Press any key to continue.
```

Tipo de dato puntero

Ejemplo de uso de operadores

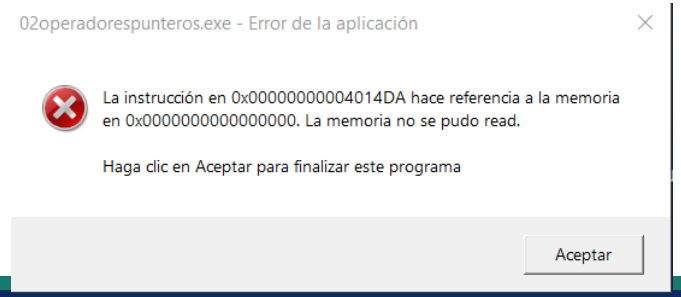
- Cuando no inicializamos el puntero apunta a una dirección que contiene un valor basura (no da error).

- Si lo inicializamos y tratamos de acceder al valor da error.

- Comprobación:

```
//Que pasa si no asignamos nada al puntero
float * ptr;
cout << *ptr;
```

```
8.9997e-039
Process returned 0 (0x0)   execution time : 1.326 s
Press any key to continue.
```




```
float * ptr=0;
cout << *ptr;
```

```
Process returned -1073741819 (0xC0000005)   execution time : 5.554 s
Press any key to continue.
```

Tipo de dato puntero

Ejemplo de uso de operadores

- Podemos manejar el valor de una variable a través de su puntero.

```
//Manejo de un valor a partir de un puntero
float *ptrx, x=5.4;
cout << "El antiguo valor de x es:" << endl;
cout << x << endl;
ptrx = &x;
*ptrx = 8.2;
cout << "El nuevo valor de x es:" << endl;
cout << *ptrx << " " << x << endl;
```

```
El antiguo valor de x es:
5.4
El nuevo valor de x es:
8.2 8.2

Process returned 0 (0x0)   execution time : 1.521 s
Press any key to continue.
```

También podemos hacer: cin >> *ptrx;

Tipo de dato puntero

DECSAI

Errores comunes con los operadores asociados a punteros

- Usar * sobre un puntero que no se ha inicializado adecuadamente puede causar un error grave en tiempo de ejecución. La dirección de memoria a la que apunta un puntero recién declarado puede contener datos importantes del sistema.
- Si usamos * sobre una variable que no es un puntero se produce un error de sintaxis.
- Si usamos * sobre un puntero inicializado a 0 obtenemos error de ejecución.

Tipo de dato puntero

¿Cuánto espacio ocupan los principales tipos de datos en memoria?

- Podemos averiguarlo usando sizeof, que nos indica el tamaño de la variable reservada.

```
#include<iostream>

using namespace std;

int main(){
    int a=5; bool b=true;  float c = 34.5;
    double d=43.23; char e = 'a';
    cout << sizeof(a) << endl;
    cout << sizeof(b) << endl;
    cout << sizeof(c) << endl;
    cout << sizeof(d) << endl;
    cout << sizeof(e) << endl;
}
```

```
4
1
4
8
1

Process returned 0 (0x0)   execution time : 0.042 s
Press any key to continue.
```

Tipo de dato puntero

¿Cuánto espacio ocupan los principales tipos de datos en memoria?

- Un puntero, como es una dirección de memoria, ocupa siempre lo mismo.

```
int *entero = &a;
bool* boolean = &b;
float * flotante = &c;
double * doble = &d;
char * caracter = &e;

cout << sizeof(entero) << endl;
cout << sizeof(boolean) << endl;
cout << sizeof(flotante) << endl;
cout << sizeof(doble) << endl;
cout << sizeof(caracter) << endl;
```

```
4
4
4
4
4

Process returned 0 (0x0)   execution time : 1.563 s
Press any key to continue.
```

Tipo de dato puntero

DECSAI

Beneficios del uso de punteros

- Permiten un uso eficiente de la memoria dinámica.
- Permiten el manejo de estructuras complejas (vectores, matrices, etc) mediante memoria dinámica.
- Manejo eficiente de funciones mediante el paso de punteros en vez de grandes estructuras de datos (lo que conllevaría la copia de toda la información en el stack).

Tipo de dato puntero

DECSAI

Punteros y const

- **Puntero no constante a un dato constante.**
  - Podemos modificar el puntero para que apunte a otro dato, pero no podemos modificar el dato constante.

```
const char * ptrS;
```

- **Puntero constante a un dato no constante.**
  - El puntero apunta siempre a la misma posición de memoria.
  - El dato de dicha posición de memoria puede variar.

```
char * const ptrS;
```

Tipo de dato puntero

DECSAI

Punteros y const

- **Puntero constante a un dato constante.**
  - El puntero apunta a la misma dirección de memoria y el valor almacenado en dicha dirección de memoria no se puede modificar.

```
const char * const ptrS;
```

- Lo que usaremos sobre todo será la primera opción: puntero no constante a dato constante ya que const nos interesará sobre todo para evitar modificar el valor de la variable.

Gestión de Memoria Dinámica

DECSAI

Introducción

- Llamamos memoria virtual a la cantidad de memoria que el sistema operativo reserva para la ejecución de cada programa.

- La memoria virtual se divide en:

  - **Segmento .text**: Almacena las instrucciones del programa.
  - **Segment .data y .bss**: Almacena las constantes y variables estáticas.
  - **User stack**: Se utiliza en la ejecución de funciones.
  - **Heap**: Memoria de uso libre (la gestiona el programador).

Gestión de Memoria Dinámica

DECSAI

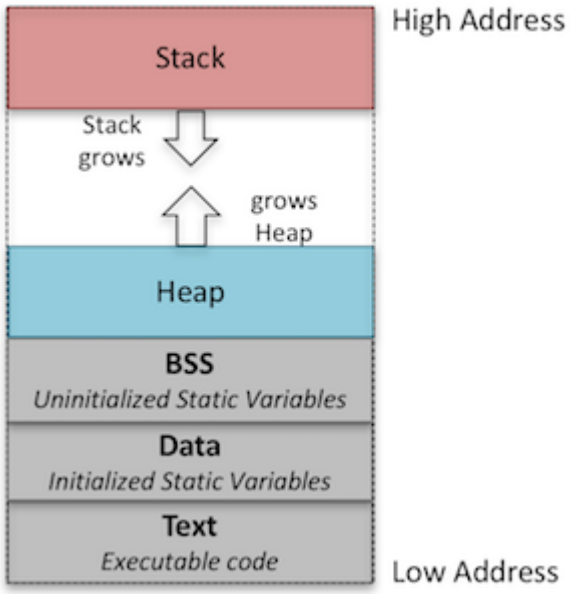
Inconvenientes de la memoria estática

- Hasta ahora sólo hemos utilizado la memoria estática.
- Almacenábamos toda la información (vectores, matrices, etc.) dentro de los segmentos .data y .bss.
- Inconvenientes:
  - La memoria estática es mucho más pequeña que la dinámica.
  - En la memoria estática debemos saber a priori cuanto van a ocupar nuestros vectores y matrices en memoria.
  - Necesitamos declarar variables MAX que ocupan espacio durante toda la ejecución del programa.

Gestión de Memoria Dinámica

Stack

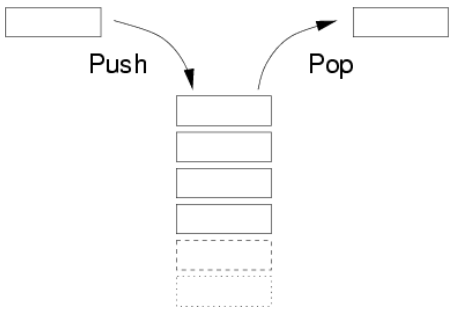
- Parte de la memoria destinada a almacenar aquellas variables de ámbito local declaradas dentro de las funciones.
- Cada vez que se llama a una función, se añade su entorno con lo que el tamaño del stack aumenta.
- Cuando una función termina de ejecutarse, se elimina su entorno.
- Es LIFO (last in, first out).
- El manejo de esta memoria la realiza el sistema operativo.

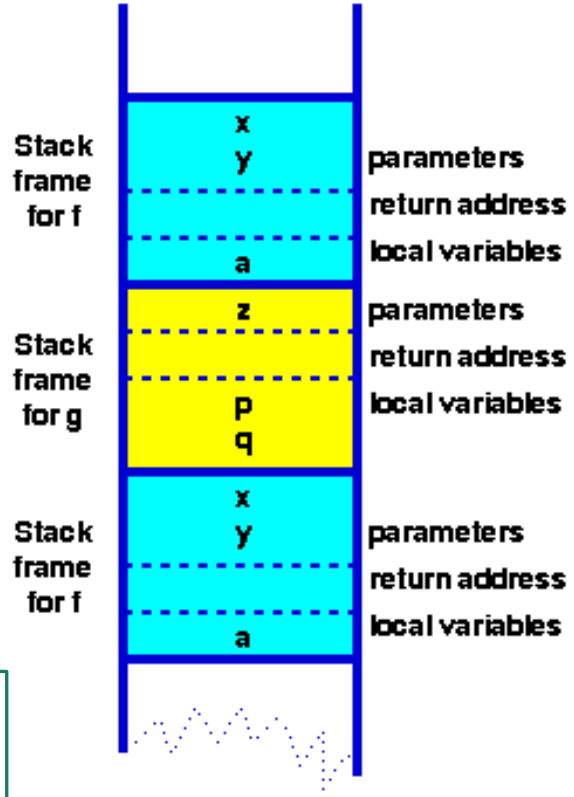


https://study.com/academy/lesson/dynamic-memory-allocation-definition-example.html

Gestión de Memoria Dinámica

Stack






```
function f(int x, int y) {
    int a;
    if ( term_cond ) return ...;
    a = .....;
    return g(a);
    }

function g(int z) {
    int p,q;
    p = ...; q = ...;
    return f(p,q);
    }
```

Si el algoritmo recursivo siguiera hasta el infinito, se puede desbordar la pila.

https://www.cs.auckland.ac.nz/software/AlgAnim/stacks.html

Gestión de Memoria Dinámica

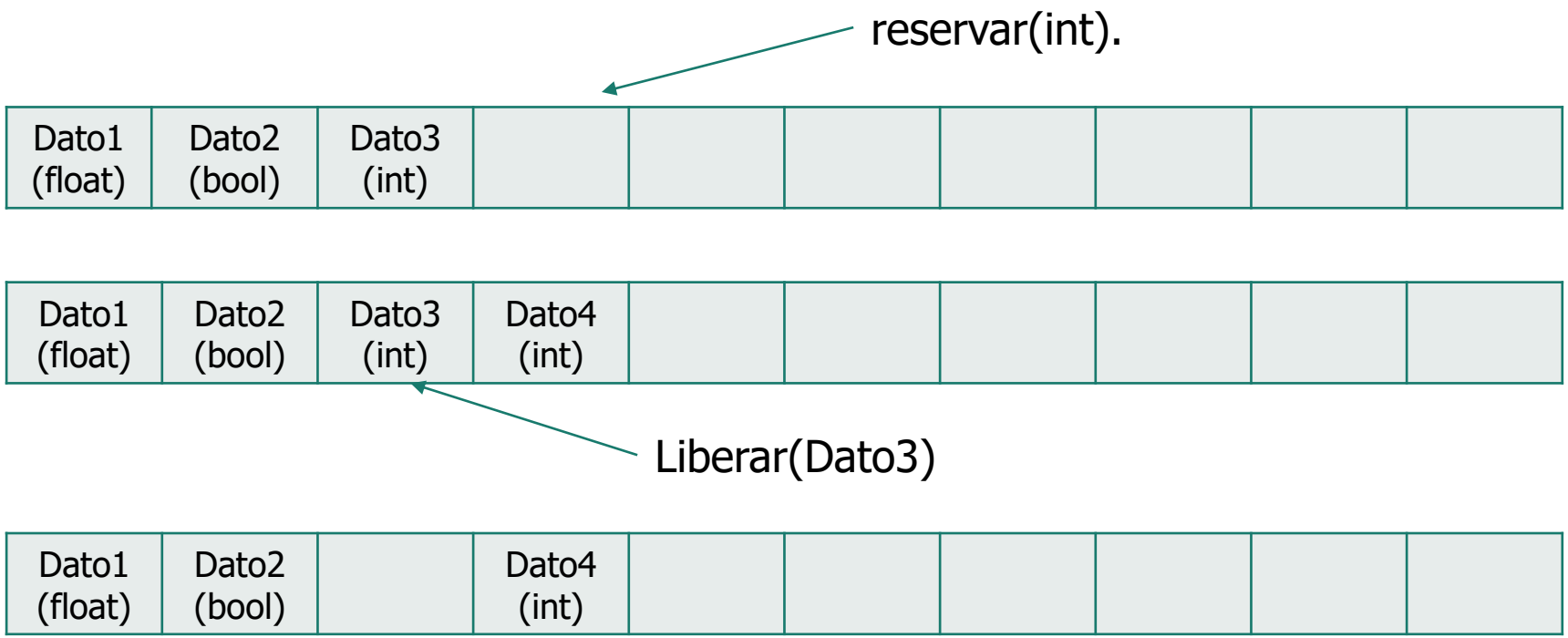
DECSAI

Heap

- Se utiliza para almacenar aquellos datos que se crean durante la ejecución del programa.
- El programador debe incluir en sus programas instrucciones específicas para reservar y liberar memoria.
- Una vez reservado un trozo de memoria, éste no puede ser usado en ninguna otra petición.
- El Heap no es infinito, con lo que un aprovechamiento adecuado de esta memoria hará que los programas sean más eficientes y ocupen menos recursos.

Gestión de Memoria Dinámica

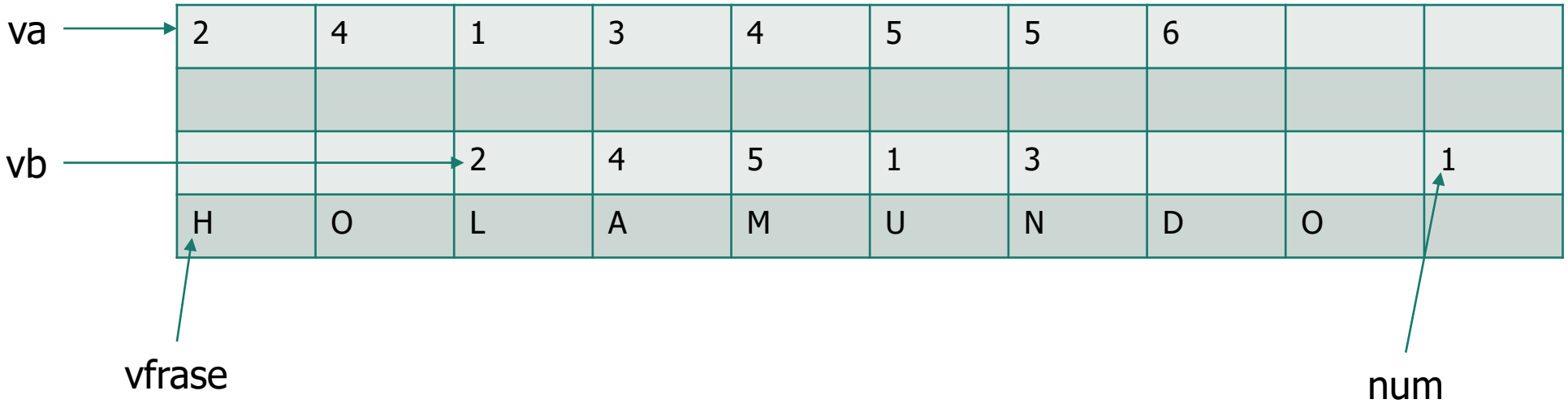
Ejemplo de manejo de memoria en el heap



Gestión de Memoria Dinámica

Uso del heap

- Para cada espacio reservado, se utiliza un puntero para su acceso.



Gestión de Memoria Dinámica

DECSAI

Uso de Stack y Heap

- El Stack y el Heap tienen tamaños limitados.
- Controlamos el tamaño ocupado por el stack teniendo en cuenta el número de variables locales utilizadas.
- El caso de que se ocupe toda la memoria disponible para el proceso el programa dará error.
- Buena práctica de programación:
  - Variables simples en stack y vectores, matrices y datos complejos en el heap.

Gestión de Memoria Dinámica

DECSAI

Datos estáticos vs datos dinámicos

- Según donde y cómo declaremos los datos distinguimos entre:

- **Datos estáticos**
  - Los declarados fuera de las funciones o como static.

- **Datos dinámicos**
  - Varían durante la ejecución del programa.
  - Variables locales o dentro de funciones→ en el stack.
  - Los reservamos nosotros → en el heap.

Gestión de Memoria Dinámica

DECSAI

Uso de static para la declaración de variables estáticas

- La variable existe durante toda la ejecución del programa.
- No es accesible fuera de su ámbito (al igual que las variables normales).
- Se reservan al principio del programa y se eliminan cuando este acaba.

```
static int a = 6;
static bool vect[6];
const static char no = 'n';
```

¿Por qué C/C++?

Feb 2026	Feb 2025	Change	Programming Language	Ratings	Change
1	1		Python	21.81%	-2.08%
2	4	^	C	11.05%	+1.22%
3	2	v	C++	8.55%	-2.82%
4	3	v	Java	8.12%	-2.54%
5	5		C#	6.83%	+2.71%
6	6		JavaScript	2.92%	-0.85%
7	10	^	Visual Basic	2.85%	+0.81%
8	15	^^	R	2.19%	+1.14%
9	7	v	SQL	1.93%	-0.93%
10	9	v	Delphi/Object Pascal	1.88%	-0.29%

¿Por qué C/C++?

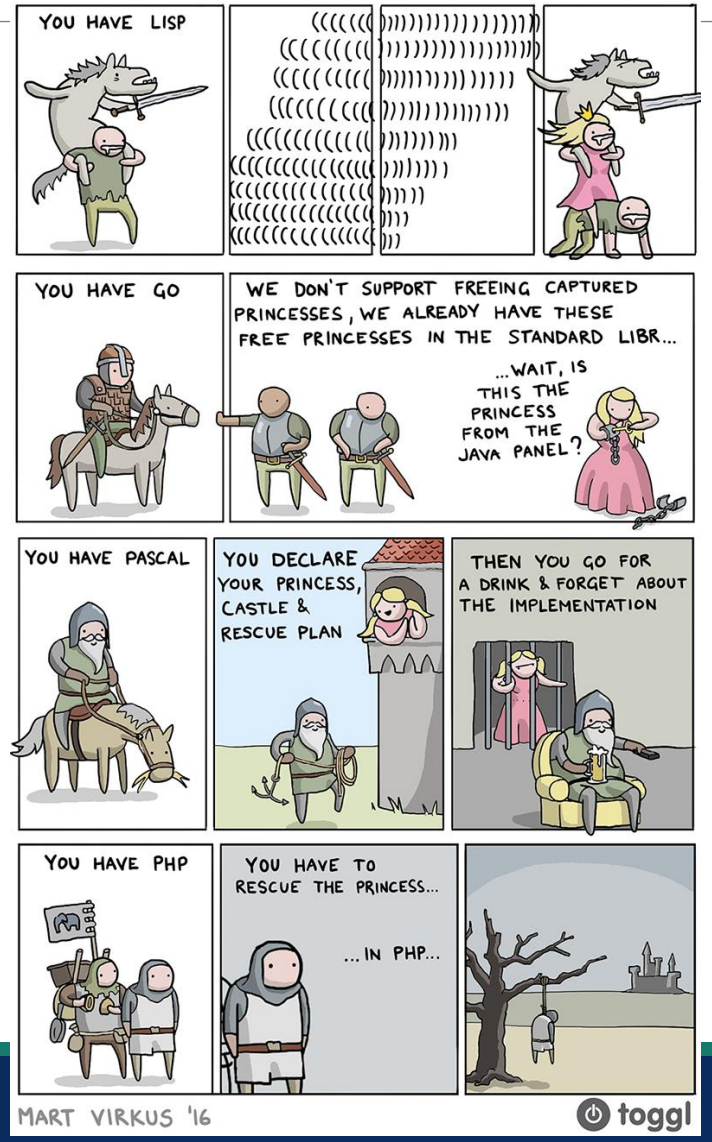
	Energy
(c) C	1.00
(c) Rust	1.03
(c) C++	1.34
(c) Ada	1.70
(v) Java	1.98
(c) Pascal	2.14
(c) Chapel	2.18
(v) Lisp	2.27
(c) Ocaml	2.40
(c) Fortran	2.52
(c) Swift	2.79
(c) Haskell	3.10
(v) C#	3.14
(c) Go	3.23
(i) Dart	3.83
(v) F#	4.13
(i) JavaScript	4.45
(v) Racket	7.91
(i) TypeScript	21.50
(i) Hack	24.02
(i) PHP	29.30
(v) Erlang	42.23
(i) Lua	45.98
(i) Jruby	46.54
(i) Ruby	69.91
(i) Python	75.88
(i) Perl	79.58

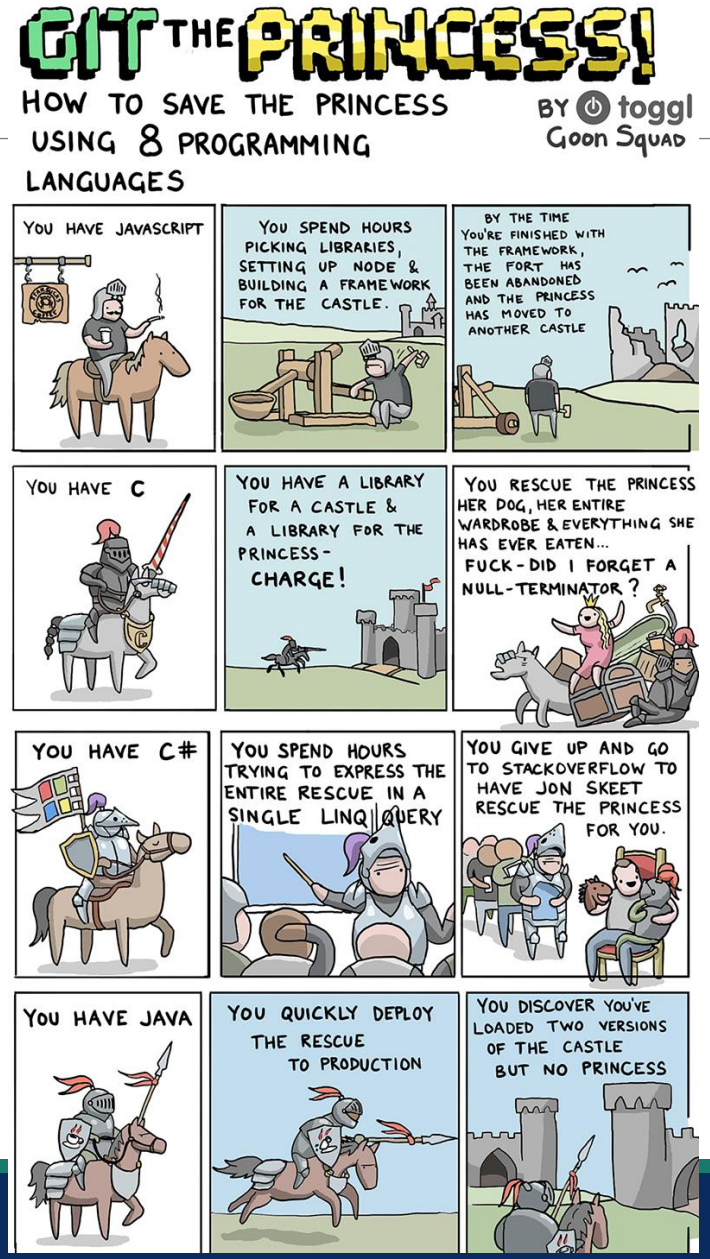
	Time
(c) C	1.00
(c) Rust	1.04
(c) C++	1.56
(c) Ada	1.85
(v) Java	1.89
(c) Chapel	2.14
(c) Go	2.83
(c) Pascal	3.02
(c) Ocaml	3.09
(v) C#	3.14
(v) Lisp	3.40
(c) Haskell	3.55
(c) Swift	4.20
(c) Fortran	4.20
(v) F#	6.30
(i) JavaScript	6.52
(i) Dart	6.67
(v) Racket	11.27
(i) Hack	26.99
(i) PHP	27.64
(v) Erlang	36.71
(i) Jruby	43.44
(i) TypeScript	46.20
(i) Ruby	59.34
(i) Perl	65.79
(i) Python	71.90
(i) Lua	82.91

	Mb
(c) Pascal	1.00
(c) Go	1.05
(c) C	1.17
(c) Fortran	1.24
(c) C++	1.34
(c) Ada	1.47
(c) Rust	1.54
(v) Lisp	1.92
(c) Haskell	2.45
(i) PHP	2.57
(c) Swift	2.71
(i) Python	2.80
(c) Ocaml	2.82
(v) C#	2.85
(i) Hack	3.34
(v) Racket	3.52
(i) Ruby	3.97
(c) Chapel	4.00
(v) F#	4.25
(i) JavaScript	4.59
(i) TypeScript	4.69
(v) Java	6.01
(i) Perl	6.62
(i) Lua	6.72
(v) Erlang	7.20
(i) Dart	8.64
(i) Jruby	19.84

Pereira, R. *et al.* Energy efficiency across programming languages: how do energy, time, and memory relate? in *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering* 256–267 (Association for Computing Machinery, New York, NY, USA, 2017). doi:10.1145/3136014.3136031.

¿Por qué C/C++?


YOU HAVE LISP
YOU HAVE GO
WE DON'T SUPPORT FREEING CAPTURED PRINCESSES, WE ALREADY HAVE THESE FREE PRINCESSES IN THE STANDARD LIBR...
...WAIT, IS THIS THE PRINCESS FROM THE JAVA PANEL?
YOU HAVE PASCAL
YOU DECLARE YOUR PRINCESS, CASTLE & RESCUE PLAN
THEN YOU GO FOR A DRINK & FORGET ABOUT THE IMPLEMENTATION
YOU HAVE PHP
YOU HAVE TO RESCUE THE PRINCESS...
...IN PHP...
MART VIRKUS '16
toggl


GIT THE PRINCESS!
HOW TO SAVE THE PRINCESS USING 8 PROGRAMMING LANGUAGES
BY toggl Goon Squad
YOU HAVE JAVASCRIPT
YOU SPEND HOURS PICKING LIBRARIES, SETTING UP NODE & BUILDING A FRAMEWORK FOR THE CASTLE.
BY THE TIME YOU'RE FINISHED WITH THE FRAMEWORK, THE FORT HAS BEEN ABANDONED AND THE PRINCESS HAS MOVED TO ANOTHER CASTLE
YOU HAVE C
YOU HAVE A LIBRARY FOR A CASTLE & A LIBRARY FOR THE PRINCESS - CHARGE!
YOU RESCUE THE PRINCESS HER DOG, HER ENTIRE WARDROBE & EVERYTHING SHE HAS EVER EATEN... FUCK - DID I FORGET A NULL-TERMINATOR?
YOU HAVE C#
YOU SPEND HOURS TRYING TO EXPRESS THE ENTIRE RESCUE IN A SINGLE LINQ QUERY
YOU GIVE UP AND GO TO STACKOVERFLOW TO HAVE JON SKEET RESCUE THE PRINCESS FOR YOU.
YOU HAVE JAVA
YOU QUICKLY DEPLOY THE RESCUE TO PRODUCTION
YOU DISCOVER YOU'VE LOADED TWO VERSIONS OF THE CASTLE BUT NO PRINCESS

Gestión de Memoria Dinámica

Uso de static en la declaración de variables estáticas

```
#include <iostream>

using namespace std;

void func() {
    //esta x solo se declara una vez
    static int x = 0;
    cout << x << endl;
    x = x + 1;
}

int main(){

    func();
    func();
    func();
```

```
0
1
2

Process returned 0 (0x0)   execution time : 2.093 s
Press any key to continue.
```

Punteros y vectores

DECSAI

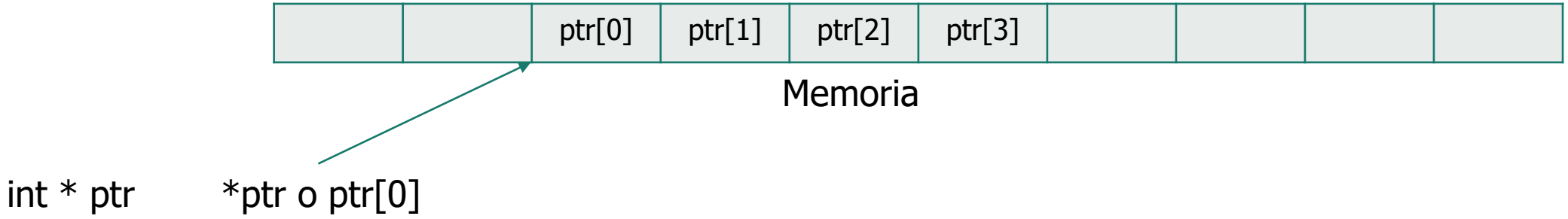
Introducción

- Hasta ahora hemos trabajado con los punteros como si sólo permitieran representar variables simples.
- Sin embargo, su principal uso será para la definición de variables compuestas (vectores, matrices, etc.) dentro del heap.
- El uso del heap para declaración de variables compuestas tiene 2 ventajas:
  - Mayor aprovechamiento del espacio (podemos ir liberando lo que no necesitemos).
  - Más cantidad de espacio que poder utilizar.

Punteros y vectores

Operador corchete en apuntadores

- El puntero apunta a una posición fija en memoria. Podemos usar el operador corchete para acceder a posiciones sucesivas.



- ¡CUIDADO!: ¡No debemos acceder a zonas de memoria que no hayamos reservado!

Punteros y vectores

Puntero a vector

- Podemos utilizar el operador corchete sobre punteros para acceder a posiciones del vector a través del puntero.
- En verdad un vector es un *puntero constante* a una posición de memoria en donde se reserva la cantidad de espacio necesaria para almacenar los elementos deseados.
- Por tanto, el corchete es un operador que trabaja sobre punteros, es decir, sobre variables que acceden a direcciones de memoria.
- El tipo de un vector/puntero, le indica al sistema qué tipo de dato está almacenado y cuando espacio debe leer a continuación.

Punteros y vectores

Ejemplo: Manejo de puntero con corchetes

```
int * puntero;
int a = 4;
puntero = &a;
cout << *puntero <<  "  " << puntero[0] << endl;
```

```
4  4

Process returned 0 (0x0)   execution time : 1.421 s
Press any key to continue.
```

Puntero[1] accedería a una posición fuera de memoria

```
int v[5] = {1,2,3,4,5};
int * ptrv = v;

for(int i = 0; i < 5; i++){
    cout << ptrv[i] << " ";
}
cout << endl;
```

```
1 2 3 4 5

Process returned 0 (0x0)   execution time : 1.392 s
Press any key to continue.
```

Punteros y vectores

Ejemplo: Manejo de puntero con corchetes

- Si hacemos que el puntero apunte a la posición 2 entonces la posición 0 será la posición 2 del vector y sucesivamente.

```
//Puntero que apunta a la posicion 2
int v2[5] = {1,2,3,4,5};
int * ptrv2 = &v2[2];

for(int i = 0; i < 3; i++){
    cout << ptrv2[i] << " ";
}
cout << endl;
```

```
3 4 5

Process returned 0 (0x0)   execution time : 2.753 s
Press any key to continue.
```

Punteros y vectores

Operadores aritméticos y punteros

- Puede accederse de forma relativa a posiciones de memoria utilizando los operadores aritméticos + y -.

- Puede usarse: +, -, ++, --, +=, -=.

```
int v[5] = {1,2,3,4,5};
int * ptrv = &v[0];
for(int i = 0; i < 5; i++){
    cout << *(ptrv+i) << " ";
}
cout << endl;
}
```

```
1 2 3 4 5

Process returned 0 (0x0)   execution time : 1.273 s
Press any key to continue.
```

ptrv[i] es igual a *(ptrv+i)

Punteros y vectores

Operadores aritméticos y punteros

- Cuando hacemos ptr+1 no significa que la dirección de memoria se incrementa en 1 valor sino que se pasa al siguiente elemento almacenado del tipo del puntero.

```
cout << ptrv << " " << ptrv+1 << endl;
```

```
0x61fee8 0x61feec
```

- Podemos restar 2 punteros entre sí y obtendremos un índice concreto:

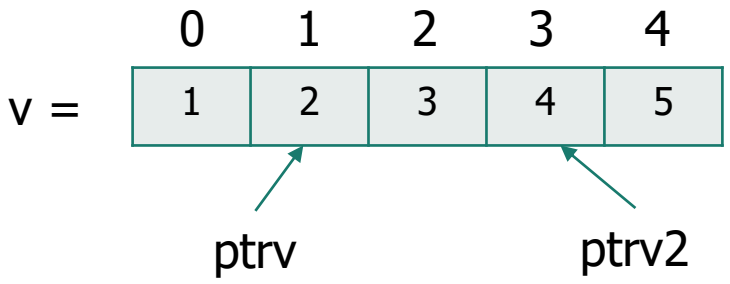
```
int v[5] = {1,2,3,4,5};
int * ptrx;
int * ptry;
ptrx = &v[1];
ptry = &v[4];
cout << v[(ptry - ptrx)] << endl;
```

```
4

Process returned 0 (0x0)   execution time : 1.296 s
Press any key to continue.
```

Punteros y vectores

Operadores relacionales y punteros

- Pueden compararse diferentes posiciones de memoria utilizando los operadores relacionales.
- Pueden usarse: <, >, <=, >=.




```
ptrv = &v[1];
int * ptrv2 = &v[3];

if(ptrv < ptrv2){
    cout << "ptrv apunta a un indice anterior a ptrv2" << endl;
}else{
    cout << "ptrv apunta a un indice posterior a ptrv2" << endl;
}
```

```
ptrv apunta a un indice anterior a ptrv2

Process returned 0 (0x0)   execution time : 1.291 s
Press any key to continue.
```

Punteros y vectores

Recorrer un array modificando el puntero vs hacerlo sin modificarlo

```
for(int * ptrit = v; ptrit < v+5; ptrit++){
    cout << *ptrit << " ";
}
cout << endl;
```

ptrit cambia a cada iteración

```
int v[5] = {1,2,3,4,5};
int * ptrv = &v[0];
for(int i = 0; i < 5; i++){
    cout << *(ptrv+i) << " ";
}
cout << endl;
}
```

ptrv apunta siempre a la misma dirección de memoria

Punteros y vectores

DECSAI

Diferencias entre vectores y y punteros

- Un vector no es un puntero. C++ los trata como conceptos diferentes.
- Podemos considerar el identificador del vector como un puntero constante.
- Todas las operaciones aritméticas sobre punteros que no modifican el valor pueden aplicarse sobre el identificador de un vector.

Punteros y vectores

Ejemplos de interacciones entre vectores y punteros

```
int v[5] = {1,2,3,4,5};
int w[5] = {5,4,3,2,1};
int * ptr = v;
ptr++;  //compila y es valido
cout << *(v+3) << endl; //valido
//v++;   //no compila
//v = w; //no compila
ptr = w; //válido, ahora el puntero apunta a w
*(v+3) = *(w+3); //valido, asigna un valor de w a una posicion de v
```

```
cout << (v < ptr) << endl; //valido, comprueba que posicion de memoria va antes
cout << v << " " << ptr << " " << w << endl; //imprime la posicion de memoria de v y de ptr que es la de w
```

Vectores en Memoria Dinámica

DECSAI

Introducción

- Los punteros pueden apuntar a cualquier zona de memoria.
- Hasta ahora hemos trabajado con variables almacenadas en el stack y en .data y .bss.
- Hemos creado variables en esas zonas y hemos usado punteros para apuntar hacia ellas.
- ¿Cómo creamos y almacenamos variables en el heap?
- Para el manejo de información en el heap utilizaremos punteros.

Vectores en Memoria Dinámica

DECSAI

Operadores new y delete

- El operador new sirve para reservar espacio de memoria para una variable en el heap.

```
<tipo> * <nombrevariable> = new <tipo> [tamaño]
```

- Algunos ejemplos:
  - int * a = new int [5];
  - bool * b = new bool[10];
  - char * c = new char[num];

Vectores en Memoria Dinámica

DECSAI

Operadores new y delete

- Importante a la hora de utilizar el operador new:
  - Puede utilizarse cualquier variable a la hora de establecer el tamaño, pero esta variable debe ser de tipo entero.
  - El tipo establecido después del new debe de coincidir siempre con el tipo de puntero utilizado.
  - Una vez reservado el espacio, este no desaparece hasta que liberemos el espacio de la memoria.

Vectores en Memoria Dinámica

DECSAI

Operadores new y delete

- Podemos liberar cualquier zona de espacio reservado en memoria usando el comando delete:

```
delete [] <variable de tipo puntero>
```

- Una vez que hemos liberado el espacio asociado a una variable, la zona de memoria a la que apunta está sin asignar.

- Podemos volver a utilizar la variable de forma segura si volvemos a usar new sobre ella.

Vectores en Memoria Dinámica

DECSAI

Operadores new y delete

- Algunos ejemplos con tipos simples.

```
int main(){
    int * a = new int[5];
    int tam = 4+6;
    float * b = new float[tam];
    float tamf = 32;
    //bool * c = new bool[tamf]; //error de compilacion
    delete [] b;
    //b[4] = 4; //mal, posible error de ejecucion
    delete[] a;
}
```

Vectores en Memoria Dinámica

DECSAI

Operadores new y delete

- Los operadores new[] y delete[] vistos son distintos a los operadores new y delete.
- Los operadores new y delete sirven para reservar y eliminar el espacio dedicado para un solo valor.
- Los operadores new[] y delete[] sirven para reservar espacio y liberar espacio en vectores.
- Por lo general, sólo utilizaremos el heap para almacenar grandes cantidades de datos con lo que no utilizaremos new y delete y guardaremos las variables simples en en stack o .data.
- Los vectores y matrices, que contienen grandes cantidades de datos, sí merece la pena guardarlos en el heap.

Vectores en Memoria Dinámica

DECSAI

Importancia de liberar el espacio reservado

- Aunque el heap tiene una gran cantidad de espacio disponible es importante que los programas sean eficientes y sólo reserven el espacio que necesiten en ese momento.
- Cuando un espacio de memoria es reservado, no puede ser usado por otro programa o proceso.
- El espacio de memoria del heap puede ir desde los GB a los TB y depende del ordenador con el que estemos trabajando.
- El uso repetido de new y delete puede ser lento, por tanto hay que reutilizar la memoria en la medida de lo posible.

Vectores en Memoria Dinámica

DECSAI

Operadores new y delete

- Error muy común: Mezclar el uso de delete[], new[], new y delete.
  - delete[] debe ir con new[] y new con delete.
- Algunos ejemplos de uso de los operadores new y delete sin corchetes:

```
a = new int;
*a = 5;
cout << *a << endl;
delete a;
```

```
double * t = new double;
*t = 23.45;
double v = 12;
*t = *t +v;
cout << *t << endl;
delete t;
```

Vectores en Memoria Dinámica

DECSAI

Operadores new y delete

- ¿Puedes encontrar el error que hay en este código?

```
int * a;
a = new int;
*a = 5;
cout << *a << endl;
delete a;
float * b;
b = new float;
*b = 7 + *a;
cout << *b << endl;
delete b;
```

¿Por qué *b no vale 12?

Vectores en Memoria Dinámica

DECSAI

Declaración en memoria dinámica de un objeto de clase

- El operador new permite la reserva en memoria dinámica de un único elemento.
- El objeto reservado puede ser liberado con delete.

```
#include <iostream>
using namespace std;

struct Punto{
    int x; int y;};

int main(){
    Punto x; //memoria estática

    Punto * y = new Punto(); //memoria dinámica
    delete y; //liberamos memoria dinámica
}
```

Vectores en Memoria Dinámica

DECSAI

Posibles errores y malas prácticas:

- No almacenamos adecuadamente el tamaño de un bloque.
- No inicializamos los punteros.
- Aplicamos delete sobre punteros sin inicializar o sin igualar a nullptr.
- No liberamos la memoria reservada.
- No accedemos correctamente a la estructura definida.

Vectores en Memoria Dinámica

Paso de vectores a funciones

- Debido a su ineficiencia, una función de C++ no puede recibir un vector entero como parámetro. En vez de ello, recibe un puntero.

```
int asignaquintaydevuelve(int c[10]){
    c[5] = 14;
    return c[5];
}

int main(){
    int v[10] = {1,2,3,4,5,6,7,8,9,10};
    cout << asignaquintaydevuelve(v) << endl;
    cout << v[5] << endl;
}
```

Mala praxis, poner int *c funciona igual y no te hace tener que definir el tamaño

```
14
14

Process returned 0 (0x0)   execution time : 0.048 s
Press any key to continue.
```

¡El vector v queda modificado para siempre!

Vectores en Memoria Dinámica

DECSAI

Paso por valor vs paso por referencia

- **Paso por valor**
  - La función recibe una copia del valor de la variable.
  - El valor de la variable se mantiene fuera de la función.
- **Paso por referencia**
  - El valor de la variable es modificado por la función y fuera de ésta se mantiene el cambio.

Vectores en Memoria Dinámica

Paso por valor vs paso por referencia

- Ejemplo

```
//aqui se cumple el paso por valor, a no cambia aunque
//cambió en la función
int suma(int a, int b){
    a = a+b;
    return a;
}
//aqui, al ser un vector, v cambia para siempre
int * sumatres(int * v, int tam){
    for(int i = 0; i < tam; i++){
        v[i]+=3;
    }
    return v;
}
```

```
int main(){
    int tam = 10;
    int a = 4;
    int b = 3;
    int c;
    c = suma(a,b);
    cout << a << " " << b << " " << c << endl;
    int * v = new int[tam];
    for(int i = 0; i < tam; i++) v[i] = i;
    int * w = sumatres(v,tam); //w y v apuntan al mismo vector
    for(int i = 0; i < tam; i++){
        cout << w[i] <<  " " << v[i] << endl;
    }
    delete [] v;

}
```

Como no sabemos a priori el tamaño del vector hay que pasarlo como parámetro.

```
4 3 7
3 3
4 4
5 5
6 6
7 7
8 8
9 9
10 10
11 11
12 12

Process returned 0 (0x0)   execution time : 1.492 s
Press any key to continue.
```

Vectores en Memoria Dinámica

DECSAI

Vectores dinámicos vs vectores estáticos en el paso a funciones

- Tanto si el vector es estático como si es dinámico, en ambos casos se pasa un puntero al vector haciendo que el paso a función sea por referencia.
- Como ya comentamos, trataremos de utilizar el heap (vectores dinámicos) cuando queramos representar muchos datos (para vectores y matrices).
- En los operadores dinámicos no necesitamos saber a priori cuál va a ser su tamaño, podemos calcularlo y luego reservar el espacio.
- Los operadores {} no sirven para asignar valores a los vectores dinámicos.

Vectores en Memoria Dinámica

DECSAI

Vectores dinámicos y clases (introducción)

- Los vectores dinámicos pueden ser utilizados dentro de las clases.
- Cada objeto generado tendrá un vector diferente.
- Se reservará el espacio mediante un new [] localizado en el constructor de la clase.
- Más adelante aprenderemos como poder crear un destructor de la clase que libere la memoria reservada.

Vectores en Memoria Dinámica

DECSAI

Vectores dinámicos y datos complejos

- Al igual que podíamos tener vectores de objetos de clases y de structs cuando utilizábamos vectores estáticos en FP con memoria dinámica también se puede.
- Para ello, hacemos: new <tipo de dato complejo> [número de elementos].
- Es interesante almacenar conjuntos de objetos y de tipos complejos en el heap ya que estos pueden contener mucha información y ocupar mucho espacio.
  - Con los objetos es posible que hasta cada elemento del vector contenga a su vez otro vector de elementos ya que pueden formar parte de los datos miembro de la clase.

Vectores en Memoria Dinámica

DECSAI

Vectores dinámicos y datos complejos

- Ejemplo de vector de structs.

```
struct Persona{
  int edad;
  string nombre;
};

int main(){
    Persona * personal = new Persona[7];
    personal[4].nombre="Paula";
    personal[4].edad = 4;
    cout << personal[4].nombre << " " << personal[4].edad << endl;
}
```

Vectores en Memoria Dinámica

Vectores dinámicos y datos complejos

- Ejemplo de vector de objetos.

```
int main(){
    Persona * personal = new Persona[7];
    for(int i = 0; i < 7; i++){
        cout << personal[i].datosPersonales() << endl;
    }
}
```

```
const int MAX = 100;
class Persona{

  int edad;
  string nombre;
  string dni;
  string cargosOcupados[MAX];
  int cargosocupados;
 public:
  Persona(){
    string dni = "00000000A";
    nombre = "-";
    edad = 0;
    cargosocupados=0;
  }

   string datosPersonales(){
    return nombre + " " + dni;
  }

};
```

Vectores en Memoria Dinámica

DECSAI

Matrices estáticas vs Matrices dinámicas

- Las matrices estáticas necesitan saber su tamaño en el momento de la declaración.
- No se puede exceder el tamaño máximo que tiene una matriz estática.
- El espacio reservado para valores estáticos es muy inferior que para dinámicos en los programas.
- Una matriz dinámica puede cambiar de tamaño. Puedes reservar más espacio y reasignarlo. No hay un límite máximo ya que puede cambiar a lo largo del programa.

Vectores en Memoria Dinámica

Declaración y uso de matrices estáticas

- Recordemos el uso y declaración de matrices estáticas.

```
#include <iostream>
using namespace std;

const int FILS = 3;
const int COLS = 4;

int main(){
    int m[FILS][COLS] = {{2,3,4,5},{1,2,3,4},{9,8,7,6}};
    for(int i = 0; i < FILS;i++){
        for(int j = 0; j < COLS;j++){
            cout << m[i][j] << " ";
        }
        cout << endl;
    }
}
```

```
2 3 4 5
1 2 3 4
9 8 7 6

Process returned 0 (0x0)   execution time : 1.523 s
Press any key to continue.
```

Vectores en Memoria Dinámica

DECSAI

Paso de matrices estáticas a funciones

- Hasta ahora siempre hemos pasado las matrices estáticas a funciones encapsulándolas en un objeto.
- Sin embargo, sí que es posible pasarlas como argumento de una función.
- Problema: Necesitamos siempre indicar el número de columnas en el parámetro de la función. Esto hace que esa función sólo sirva para una matriz con el número de columnas indicado.
- Este problema se resuelve con el uso de matrices en memoria dinámica.

Vectores en Memoria Dinámica

Paso de matrices estáticas a funciones

<tipo> <nombreFuncion (<tipo> <nombre> [][<numfilas>])

```
const int MAX = 100;

void imprimeMatriz(int v[][MAX], int numfils, int numcols){
    for(int i = 0; i < numfils; i++){
        for(int j = 0; j < numcols; j++){
            cout << v[i][j] << " ";
        }
        cout << endl;
    }
}

int main(){
    int v[MAX][MAX] = {{1,2,3},{4,5,6},{7,8,9}};
    imprimeMatriz(v,3,3);
}
```

Vectores en Memoria Dinámica

DECSAI

Matrices

- Podemos usar los punteros para reservar matrices.
- Hay varias maneras de hacer la reserva. Ahora mismo mostraremos una. Las demás las veremos más adelante.
- Para reservar una matriz hay que declarar un puntero a puntero:
- Ejemplos:
  - int ** matriz;
  - bool ** matriz;

Vectores en Memoria Dinámica

DECSAI

Matrices

- Código para crear una matriz y reservar la memoria necesaria.

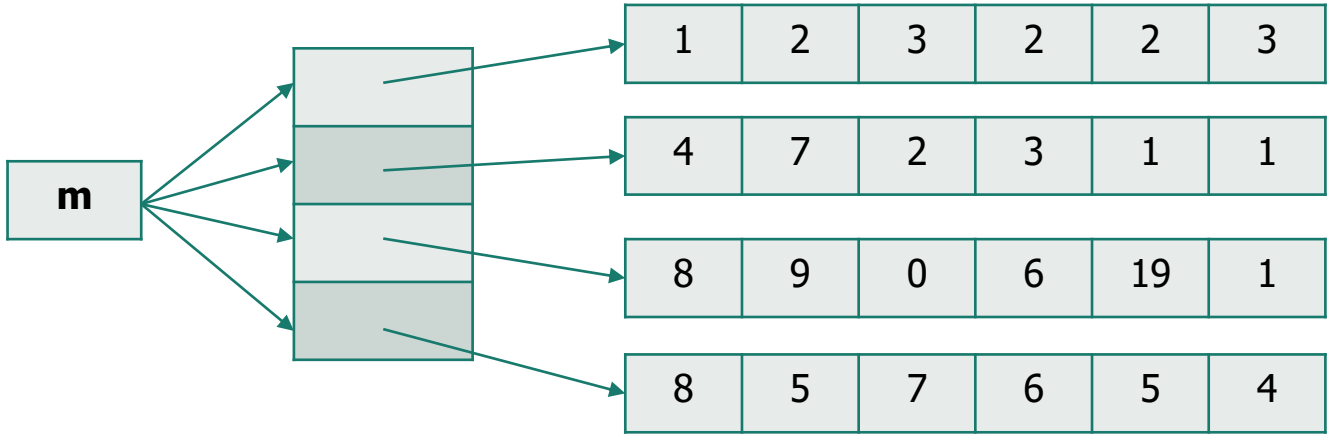
```
int ** reservaMemoriaMatriz(int fils, int cols){
    int ** m;
    m = new int * [fils];
    for(int i = 0; i < fils; i++){
        m[i] = new int[cols];
    }
    return m;
}
```

```
int main(){
    int fil = 5;
    int col = 5;
    int ** matriz = reservaMemoriaMatriz(fil,col);
    for(int i =0; i < fil; i++){
        for(int j =0; j < col; j++){
            matriz[i][j] = 0;
            cout << matriz[i][j] << " ";
        }
        cout << endl;
    }
}
```

Vectores en Memoria Dinámica

Matrices

- Representación gráfica de la reserva de memoria realizada.



Matriz de 4 filas y 6 columnas

Vectores en Memoria Dinámica

DECSAI

Matrices

- Código para liberar la memoria de una matriz.

```
void liberaMemoria(int ** m, int fils){
    for(int i = 0; i < fils; i++){
        delete [] m[i];
    }
    delete [] m;
}
```

```
liberaMemoria(matriz,fil);
```

- Recorremos cada una de las posiciones del puntero a punteros y vamos liberando cada conjunto de espacio de cada puntero.

Vectores en Memoria Dinámica

Paso de parámetros con matrices

- Ejemplo:

```
void imprimeMatriz(int **v, int numfils, int numcols){
    for(int i = 0; i < numfils; i++){
        for(int j = 0; j < numcols; j++){
            cout << v[i][j] << " ";
        }
        cout << endl;
    }
}

int main(){
    int maxim = 3;
    int ** v = reservaMemoriaMatriz(maxim,maxim);
    int k=0;
    for(int i = 0; i < maxim;i++){
        for(int j = 0; j < maxim;j++){
            v[i][j] = k;
            k++;
        }
    }
    imprimeMatriz(v,maxim,maxim);
    liberaMemoria(v,maxim);
}
```

Vectores en Memoria Dinámica

DECSAI

Ventajas del uso de matrices en memoria dinámica

- Podemos conseguir una implementación más o menos eficiente según el tipo de implementación de reserva de memoria que hagamos.
- No necesitamos indicar el tamaño de las columnas cuando las pasamos a una función. Basta con pasar el puntero a puntero **.
- En el heap hay mucho espacio por lo que podemos tener matrices mucho más grandes que si las reserváramos con memoria estática.
- Al estar definidas mediante un puntero nunca se pasan a las funciones por valor. Puede modificarse su valor a no ser que usemos const.

Vectores en Memoria Dinámica

DECSAI

Ejemplos con matrices

- Suma 2 matrices.
- Calcular traspuesta de una matriz.
- Matrices de strings.